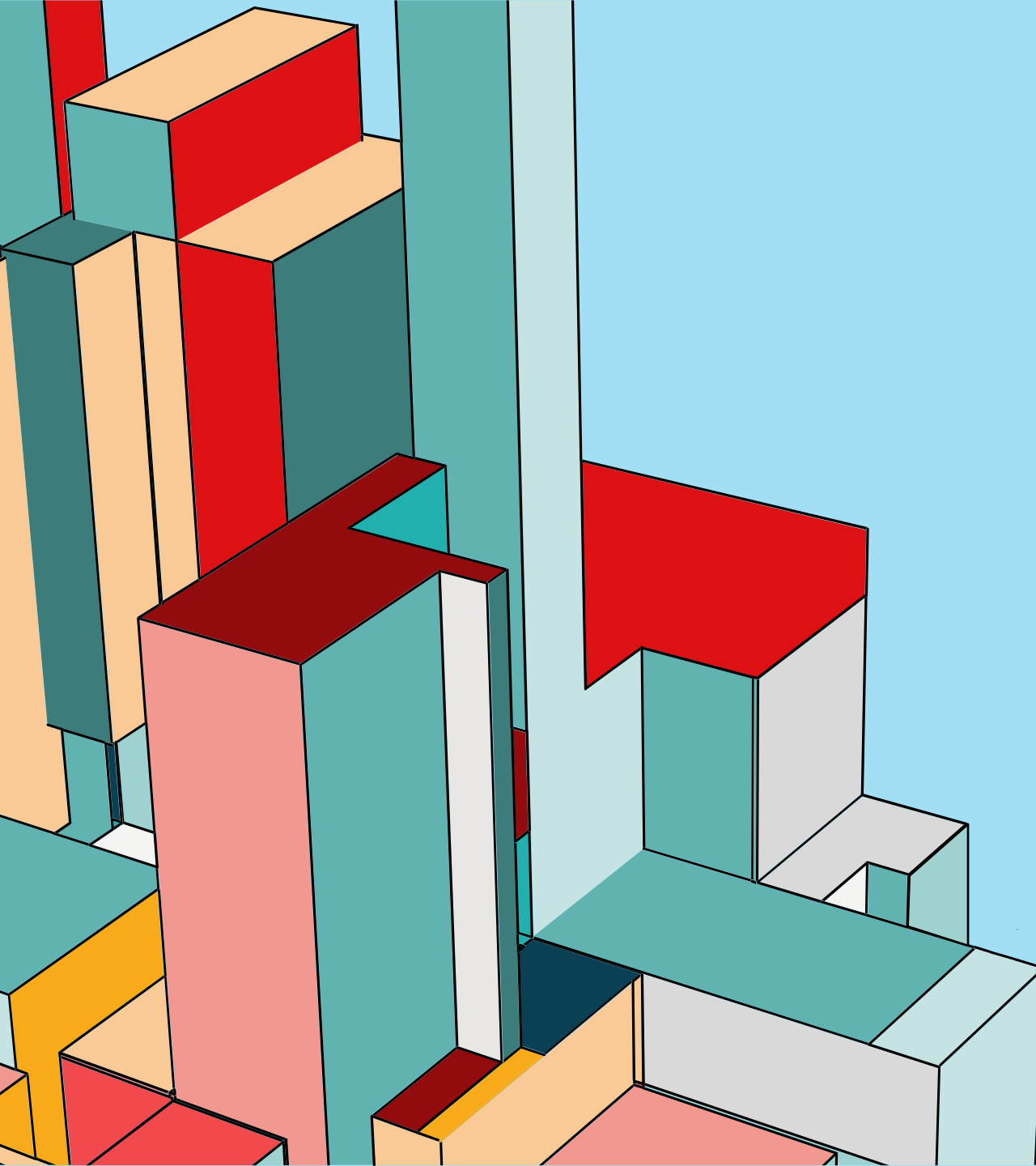
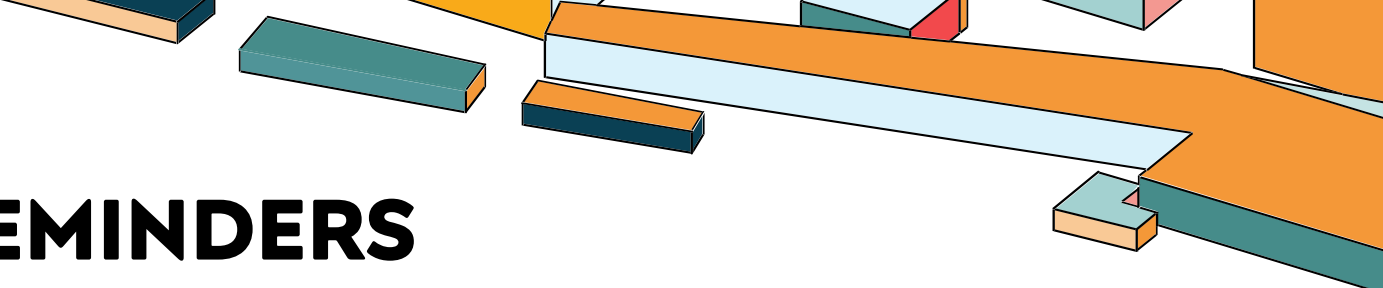




LECTURE 25 **NP-COMPLETE PROBLEMS II**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi



ANNOUNCEMENTS / REMINDERS

- Exam 3 now scored & scanned; grades posted soon.
- HW 11 due **Wednesday**, 4/29
- Lab sessions next Wednesday are essentially extended office hours (and optional) – there will be no lab activity.
- Final exam Monday 5/4 at 11:30am in DCC 308/318
 - Accommodations will start EARLIER – see your e-mail.
- Course evaluations (assessment.rpi.edu)
 - Currently 113 completed. 170+ for bonus points on final.
 - Due date Thursday 4/30.

REDUCTION TO SHOW RELATIVE DIFFICULTY

We have used reduction a number of times in this course already.

All of the previous instances have been targeted at "easy" problems: We have a fast solver for $\mathfrak{D}$, and then we show $\mathfrak{C} \leq_P \mathfrak{D}$, which immediately gives us a fast solver for $\mathfrak{C}$. ($\mathfrak{D} \in P \Rightarrow \mathfrak{C} \in P$).

In our next couple of classes, we'll be using reduction in the other direction: We know (or at least suspect) that $\mathfrak{C}$ cannot be solved quickly. Then when we show $\mathfrak{C} \leq_P \mathfrak{D}$, we conclude that $\mathfrak{D}$ also cannot be solved quickly. ($\mathfrak{C} \notin P \Rightarrow \mathfrak{D} \notin P$)

In particular, when $\mathfrak{C}$ is *NP*-hard then $\mathfrak{D}$ must also be *NP*-hard.

(*NP*-hard is just the difficulty portion of the *NP*-complete definition. A problem $\mathfrak{D}$ is *NP*-complete when it is *NP*-hard and $\mathfrak{D} \in NP$.)

ENTER RICHARD KARP...

In 1972, Karp presented "[Reducibility Among Combinatorial Problems](#)" at IBM's symposium on Complexity of Computer Computations at Watson Labs in Yorktown Heights.

"We show that a large number of classic unsolved problems of covering, matching, packing, routing, assignment and sequencing are equivalent, in the sense that either each of them possesses a polynomial-bounded algorithm or none of them does."

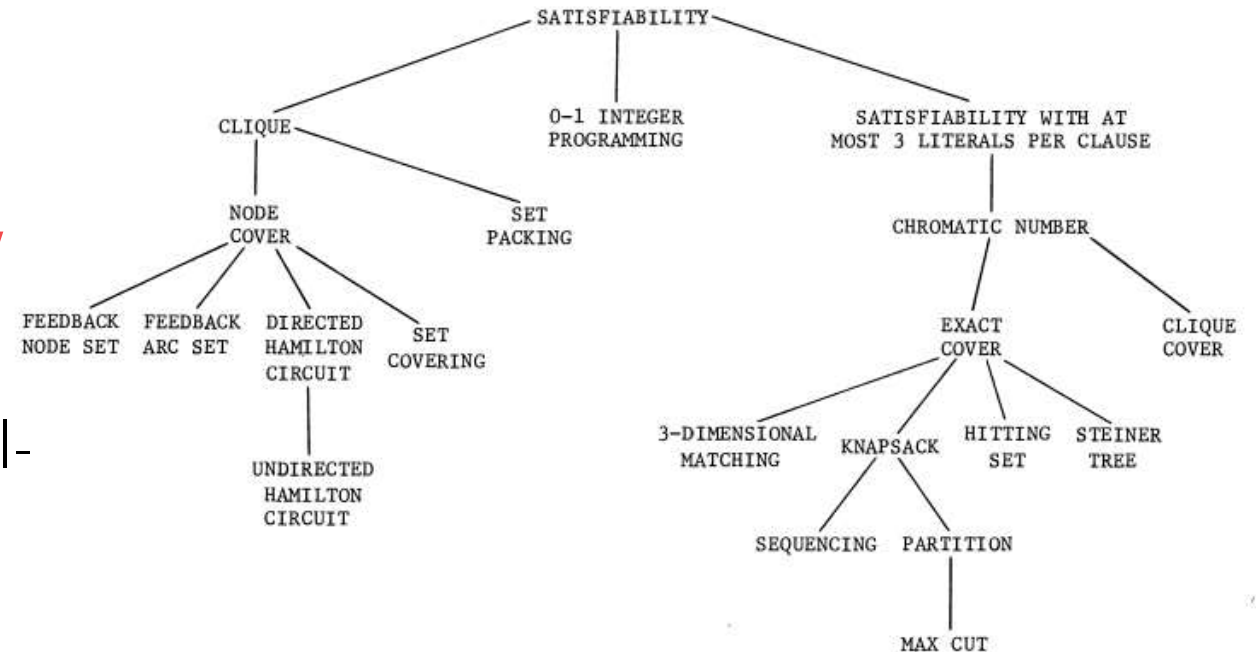


FIGURE 1 - Complete Problems

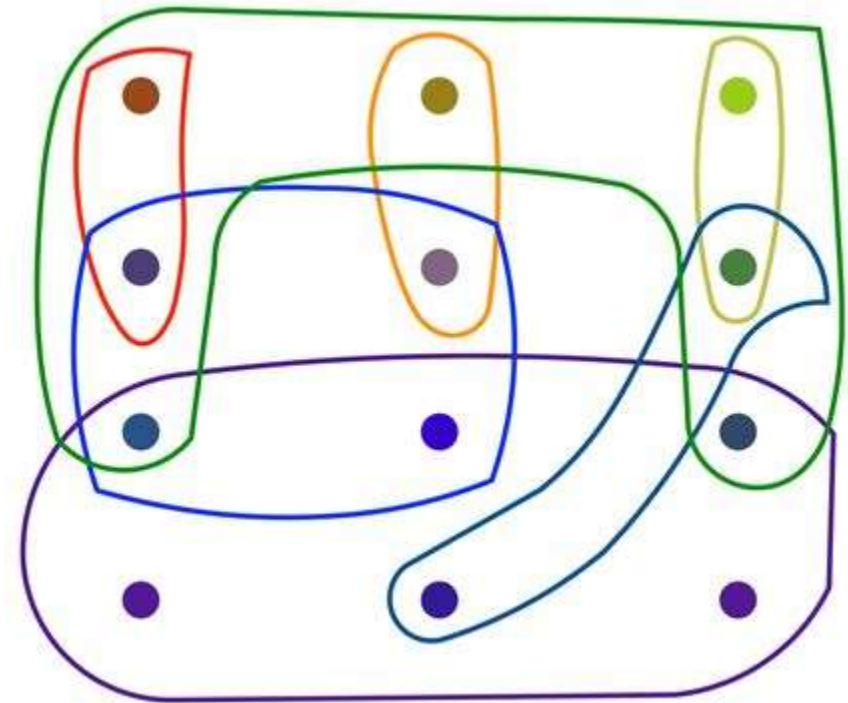
PROOF TECHNIQUE: GENERALIZATION

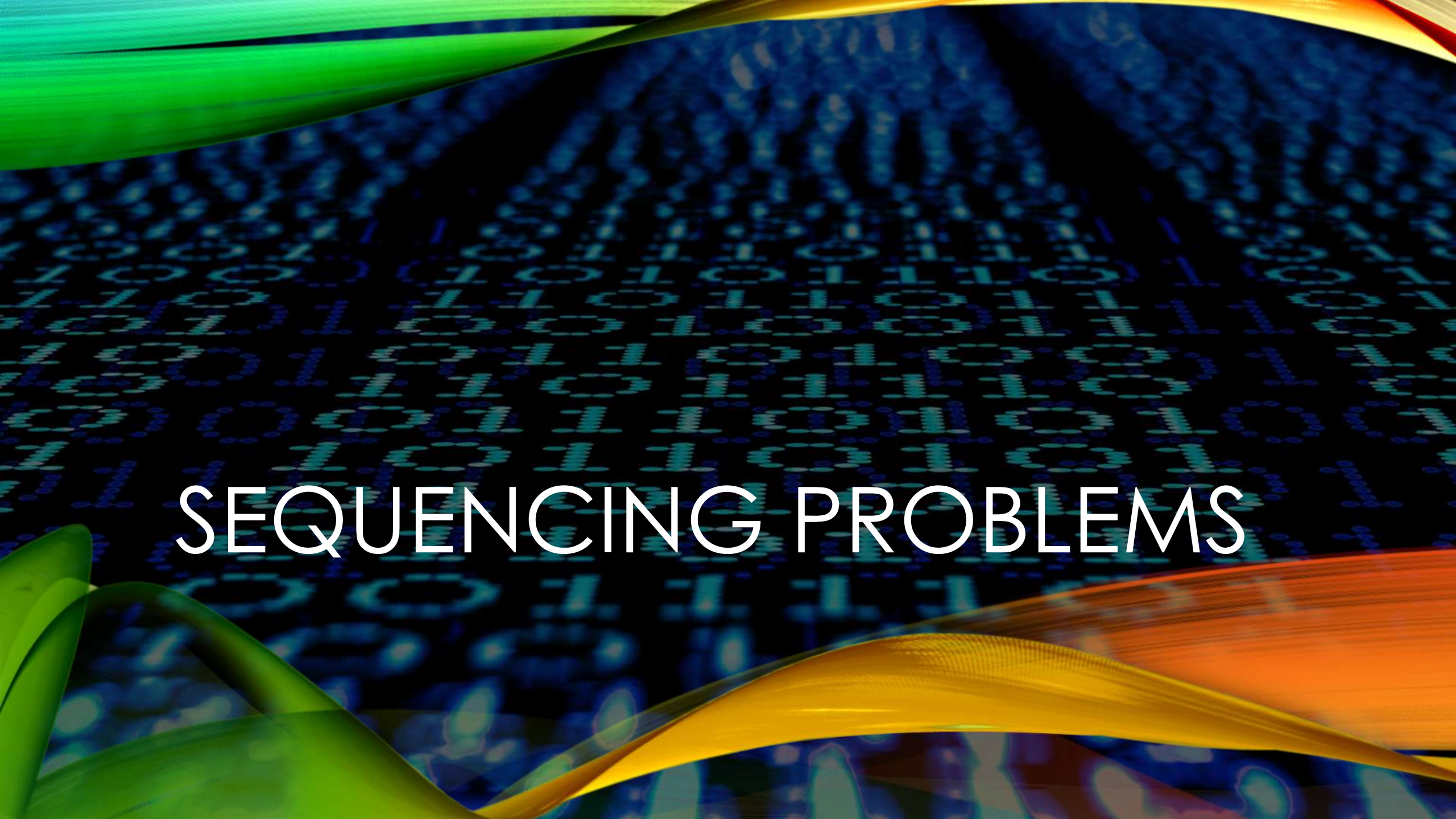
The *Set Packing* problem: Given some collection of objects U and a series of subsets of those objects $S_1, S_2, \dots, S_n \subseteq U$, find the largest collection of the S_i that have no overlap.

Set Packing is a *generalization* of Independent Set – in other words, no transformation is needed to turn IS into SP.

The vertices of the IS graph form the set U in the SP problem. The S_i are then the neighborhoods of each v_i – the nodes that are reachable from v_i plus v_i itself.

Immediately, we get $IS \leq_p SP$.



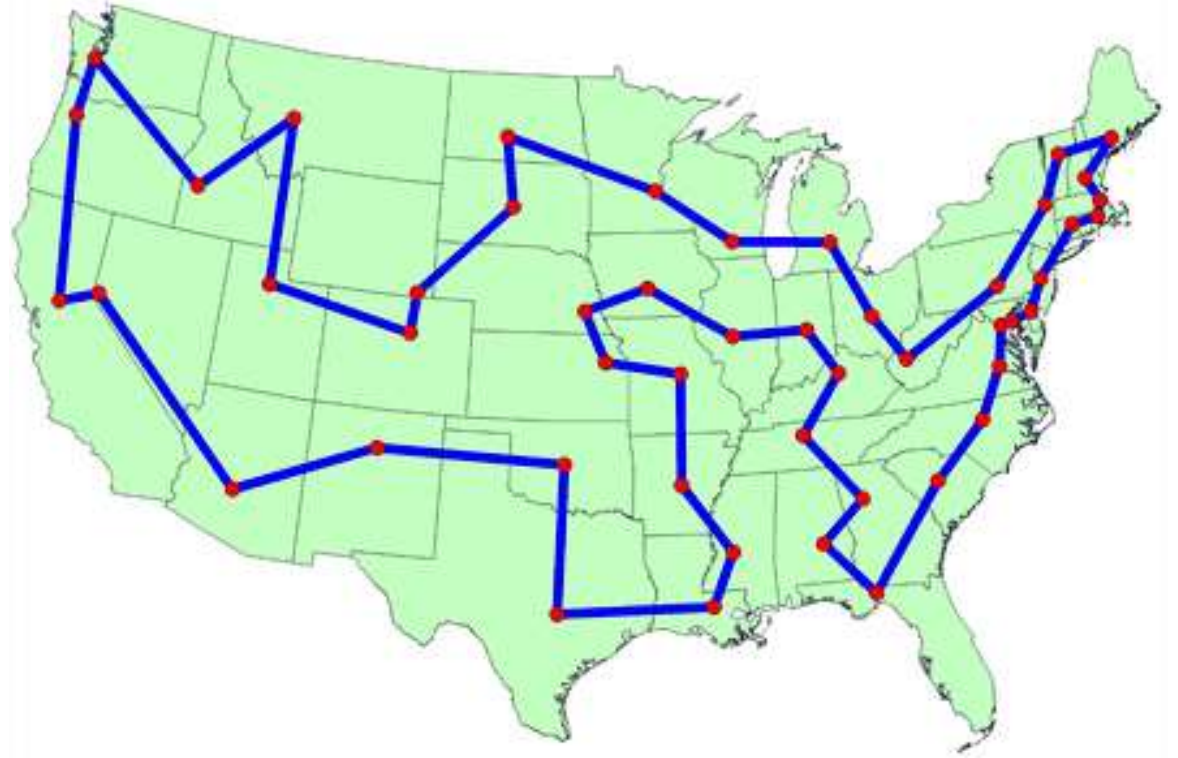
The background features a dark blue field filled with glowing green and yellow binary digits (0s and 1s) arranged in a grid-like pattern. Overlaid on this are several flowing, translucent ribbons in shades of green, yellow, and orange, creating a sense of dynamic movement. The text 'SEQUENCING PROBLEMS' is centered in a clean, white, sans-serif font.

SEQUENCING PROBLEMS

TRAVELING SALESMAN PROBLEM

The quintessential sequencing problem (and arguably the quintessential *NP*-complete problem) is the Traveling Salesman Problem.

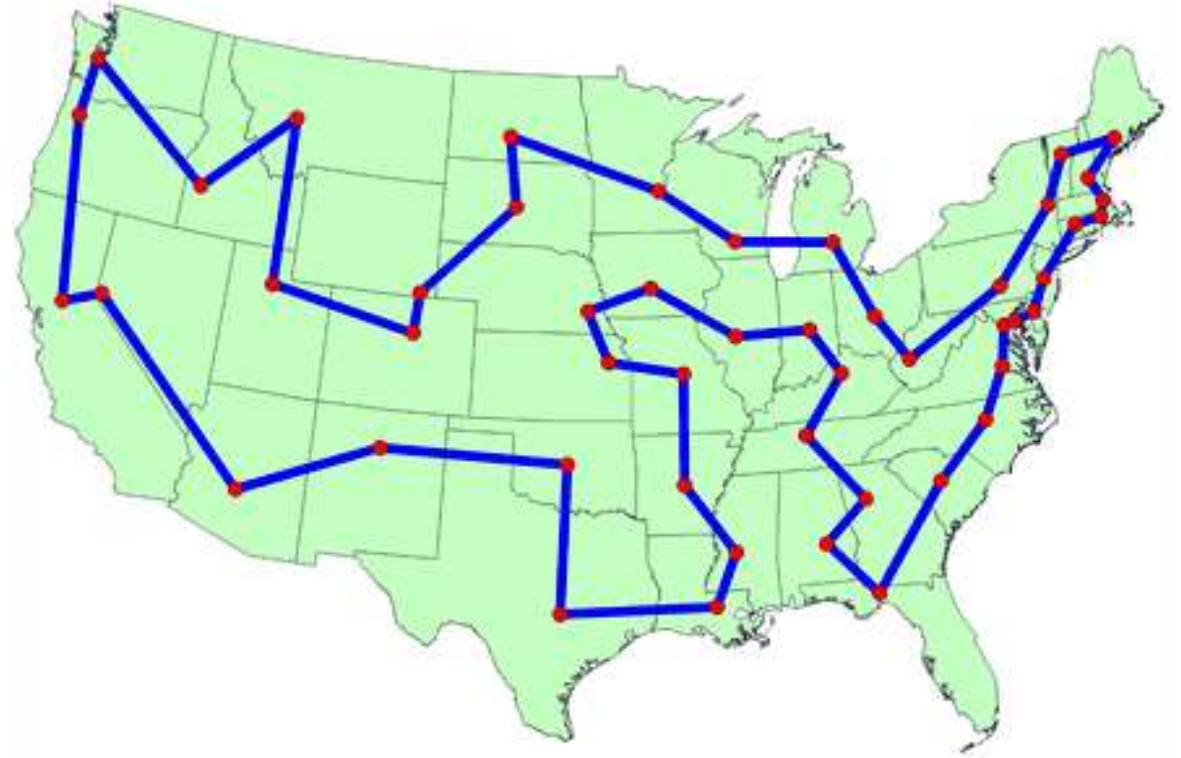
Given a set of points in the plane, find a path that visits them all, starting and ending at the same point, with total distance $\leq d$.



GENERALIZED TRAVELING SALESMAN

We don't even need to stick to Euclidean distances, nor even symmetric ones. (It takes longer to get into NYC than to get out.)

Generalization: Given a set of n "cities" and an $n \times n$ matrix of "distances" between each pair of them, find a sequence of visits to all n cities that starts and ends at the same point and has total distance $\leq d$.

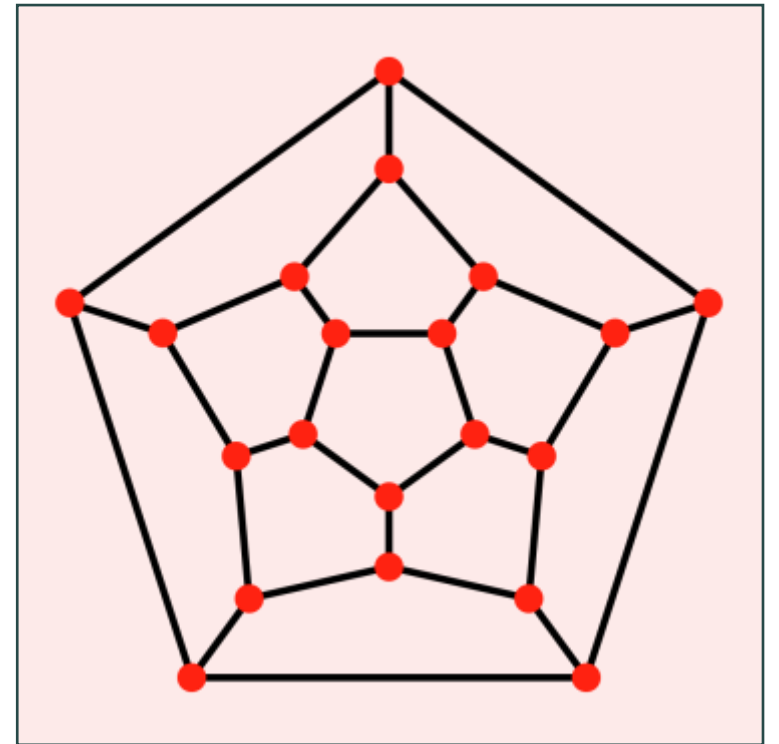


A RELATED GRAPH PROBLEM

The *Hamiltonian Cycle* problem asks if there is a path in a graph that starts and ends at the same vertex and visits every other vertex exactly once.

HC is *NP*-complete.

Proving HC is in *NP* is easy: A proposed solution is simply a permutation of V , and we check that each consecutive pair (and the first and last) have an edge between them.



REDUCTION FOR HCP

Karp used a reduction from Vertex Cover. We will examine a “direct” reduction from SAT.

Both of them qualify as “gadget” proofs, where we build a complex graph based off of the instance of the problem we are reducing to. This graph has a *gadget* (a subgraph with particular properties) for each element of the source problem. I find the SAT reduction more intuitive.

Critical lemma: Directed HCP $=_p$ Undirected HCP (Why?)

We’ll work with the directed case here.

Key question: How do we turn an instance of SAT into a graph, where a Hamiltonian Cycle will give us a SAT solution?

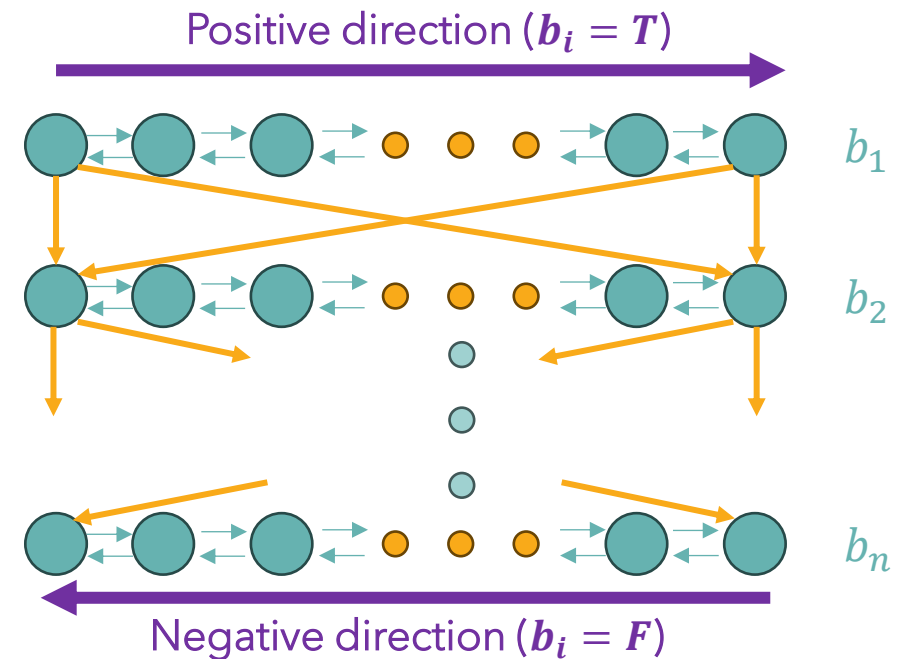
Well, there are 2^n possible assignments to n Boolean variables... can we make a graph that has 2^n paths through it?

IMPORTANT IDEA #1: TWO-WAY PATHS

We can set up a row of vertices for each b_i , where we traverse the row left to right if b_i is TRUE and from right to left if b_i is FALSE.

Note that turning around in the middle of a row leaves some nodes unvisited and thus doing so is not useful for finding a Hamiltonian cycle.

Since the variables can be assigned independently, we next add edges from both ends of one row to both ends of the next.

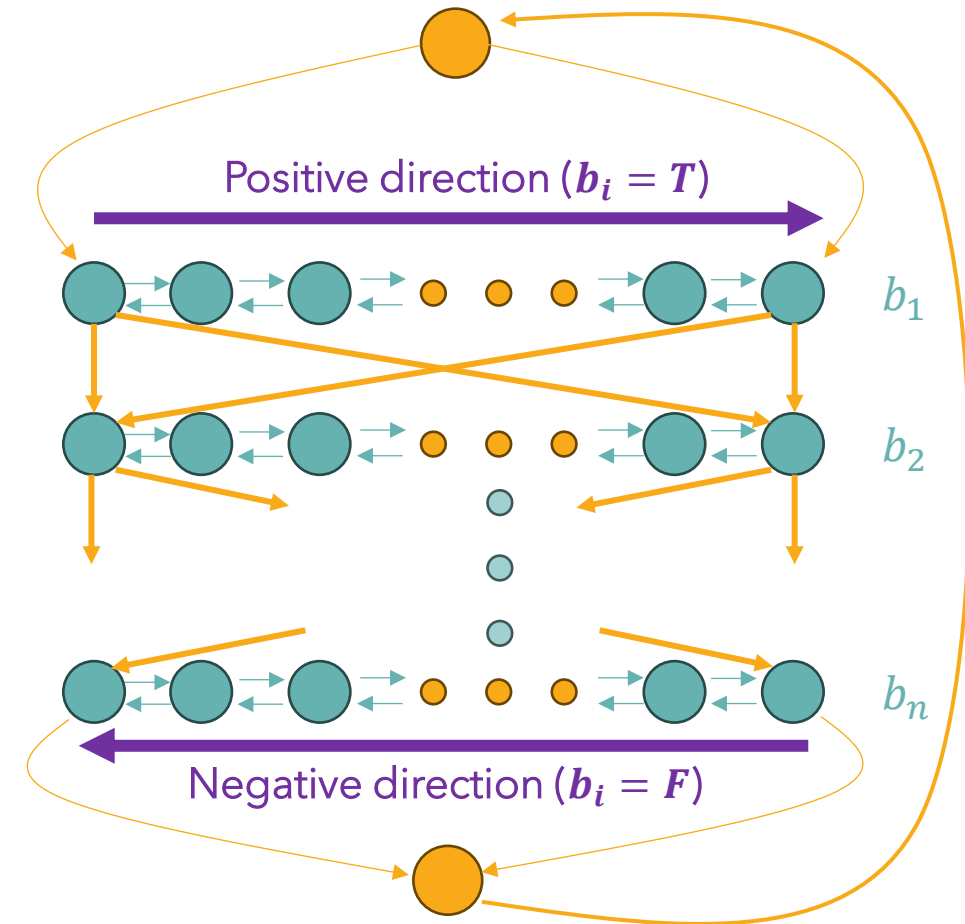


IMPORTANT IDEA #1: TWO-WAY PATHS

Next, we add source and sink nodes.

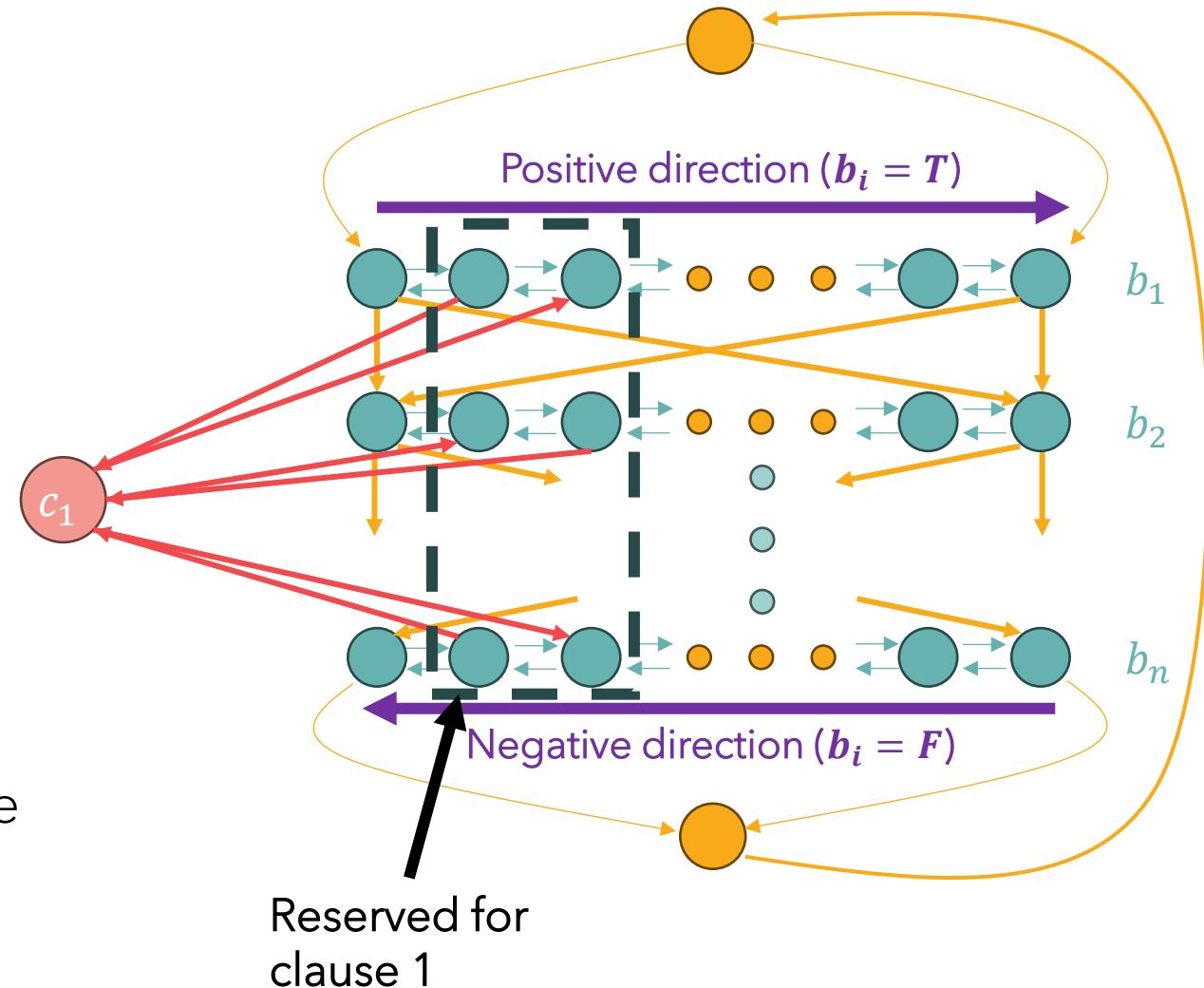
The source node connects to both ends of the b_1 row; both ends of the b_n row connect to the sink node, and we add a single edge from the sink to the source.

Now any Hamiltonian Cycle corresponds to a particular assignment of the b_i ; there are 2^n possible cycles.



IMPORTANT IDEA #2: CLAUSE NODES

- Imagine clause 1 contains:
 - $(b_1 \vee \neg b_2 \vee b_n)$
- We add a node for the clause, with links out of and into the appropriate rows, in the appropriate direction.
- We reserve the nodes on either end, and otherwise, each row has two nodes per clause (out and back into the row).
 - Technicality: We require that clauses not have duplicate terms. (Otherwise we'd need more nodes per clause.)

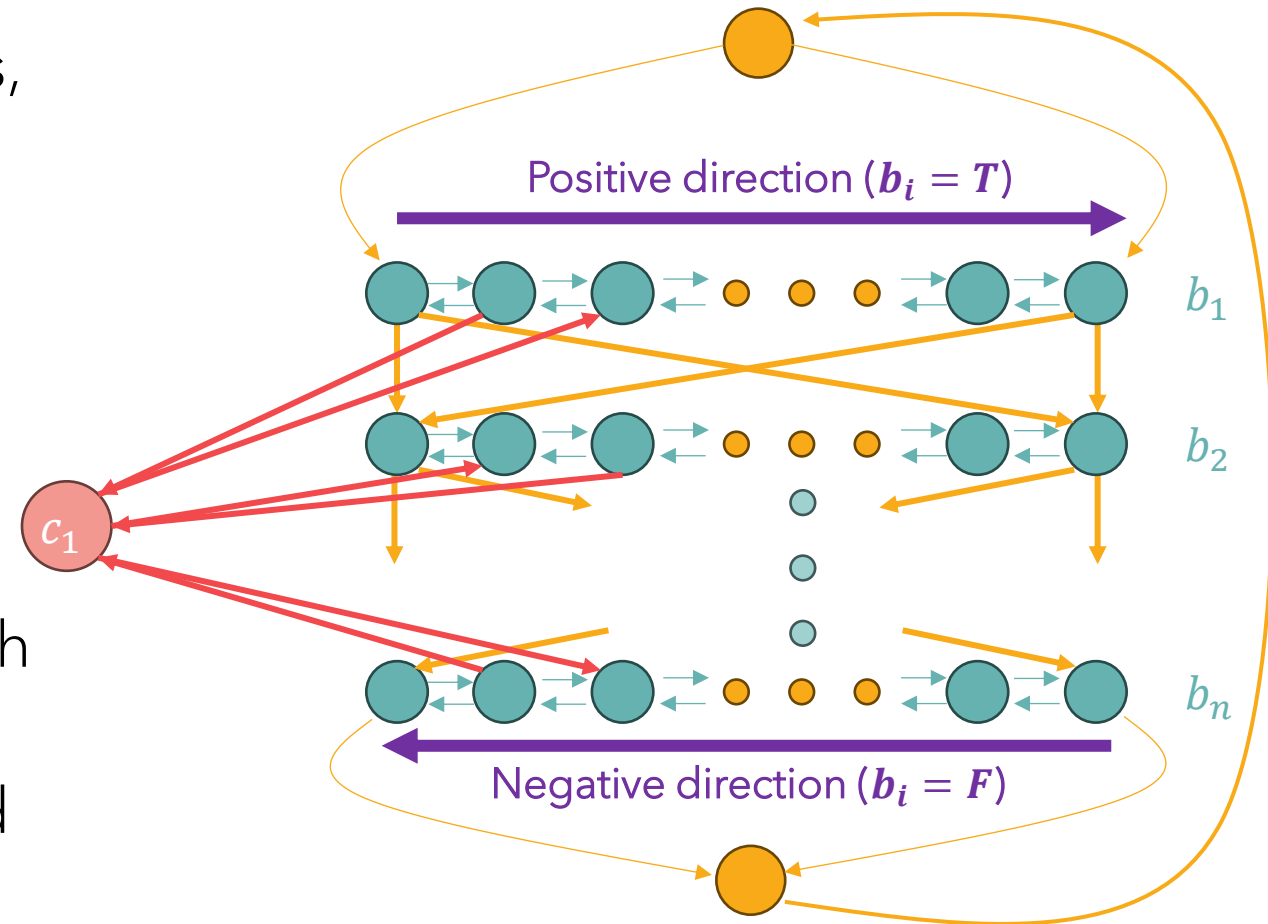


IMPORTANT IDEA #2: CLAUSE NODES

In order to hit all of the clause nodes, a Hamiltonian cycle must be traversing the row in the “correct” direction for at least one of the variables in the clause.

Intuitively, a satisfying assignment tells us how to find the Hamiltonian cycle and vice versa: by specifying the direction we should traverse each row.

Thus, $\text{SAT} \leq_p \text{Hamiltonian Cycle}$, and HC is *NP*-complete.



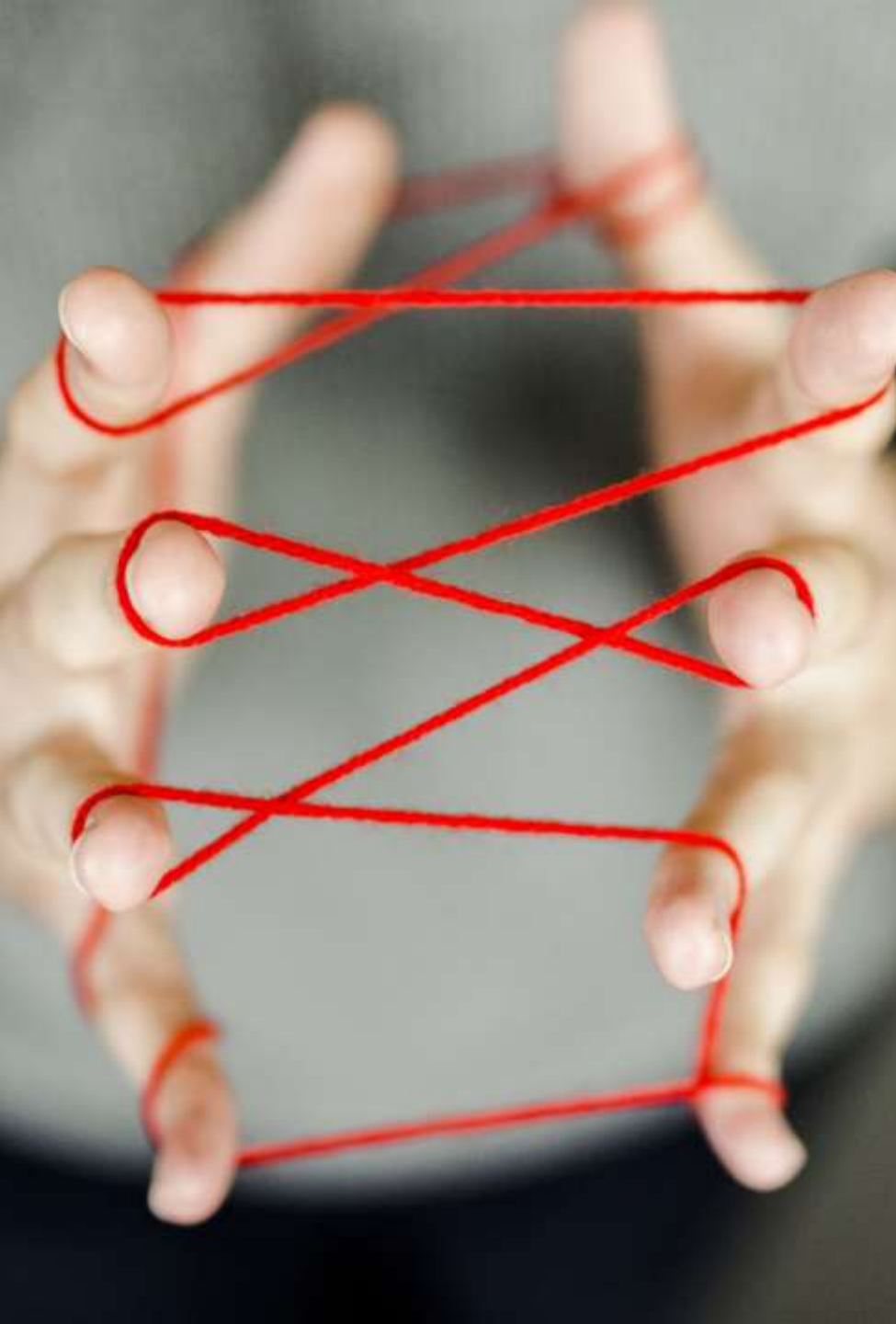
TRAVELING SALESMAN IS ALSO NP-COMPLETE

Traveling Salesman is in NP – the verifier simply ensures that the length of the proposed cycle is less than the target d .

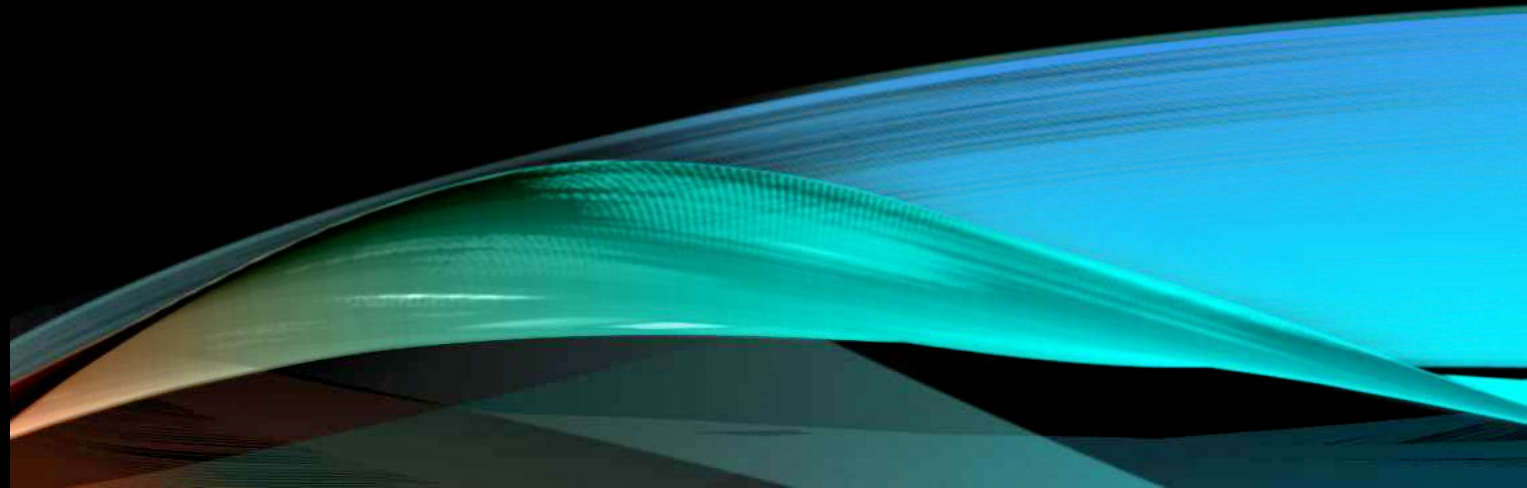
To show that $HC \leq_p TS$, take any directed graph $G(V, E)$ and create a “city” for each $v \in V$. Let the distance from city i to city j be 1 if $(v_i, v_j) \in E$ and 2 otherwise.

G has a Hamiltonian Cycle if and only if this instance of generalized TS has a tour of length n .

“Classic” TS (i.e. when limited to points on a Euclidean plane) is also NP -complete, but the reduction is trickier.



PARTITIONING PROBLEMS



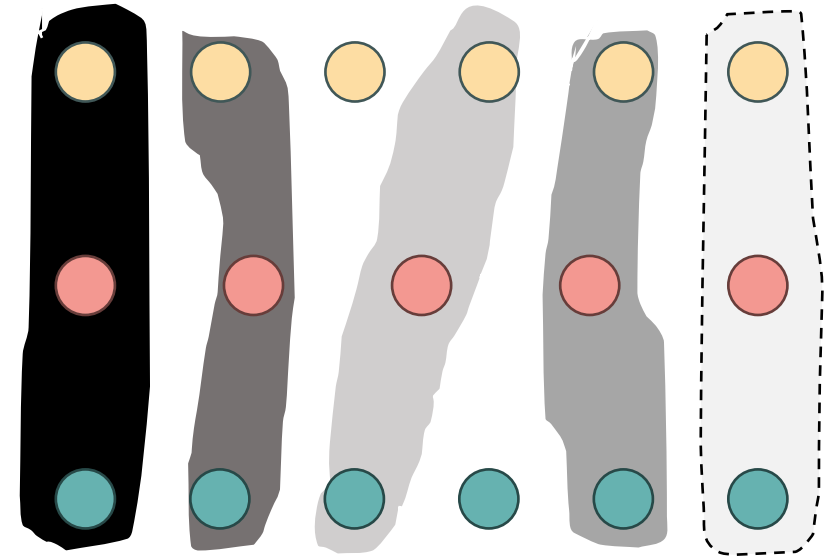
3-DIMENSIONAL MATCHING

We have previously seen a number of variations on *bipartite matching*.

In particular, we know that finding a maximum bipartite matching is reducible to Max-Flow.

What if we have *three* disjoint sets and want to match up one item from each of the three?

... turns out, that's a harder problem.

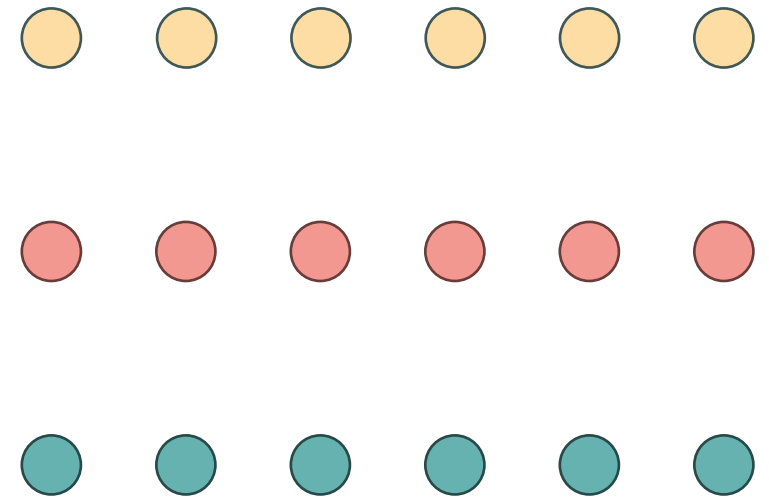


3-DIMENSIONAL MATCHING

The usual setup is that we are given three disjoint sets that are all size n , along with a set T of ordered triples containing one element from each set, and we seek a set of n triples that forms a *perfect matching*.

Note that 3DM is a special case of both Set Cover and Set Packing, so we immediately have $3DM \leq_p SC$ and $3DM \leq_p SP$...

... but these relations are the wrong direction to prove anything about 3DM.



PROVING 3DM IS NP-COMPLETE

The reduction is once again from 3-SAT, which means we'll be taking a 3-SAT instance and creating three disjoint sets and the available set of triples in such a way that a perfect matching corresponds to a satisfying 3-SAT assignment.

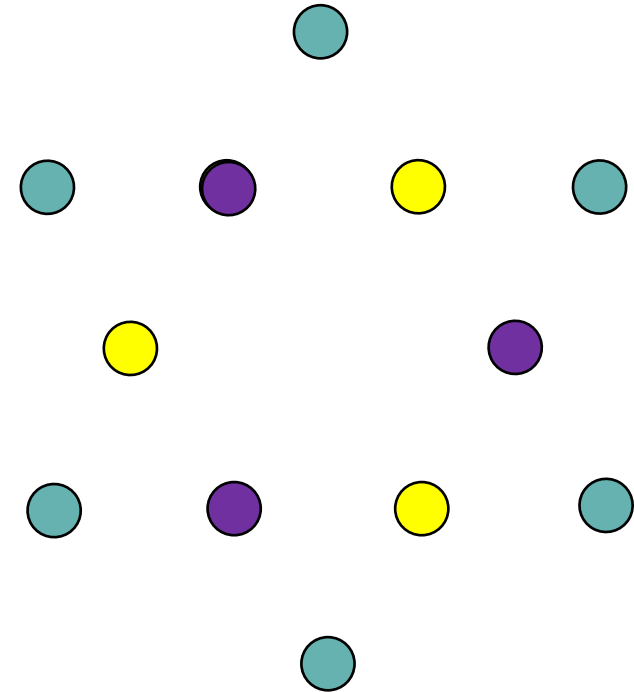
Reminder: a 3-SAT instance in standard form has n Boolean variables and k clauses with three terms each.

The first part of our reduction involves building a gadget for each variable that looks like a $2k$ -pointed star...

3DM: THE VARIABLE GADGET

For each variable, we create $4k$ objects. (Remember, k is the number of clauses.)

Our example here would be for a problem with 3 clauses.



3DM: THE VARIABLE GADGET

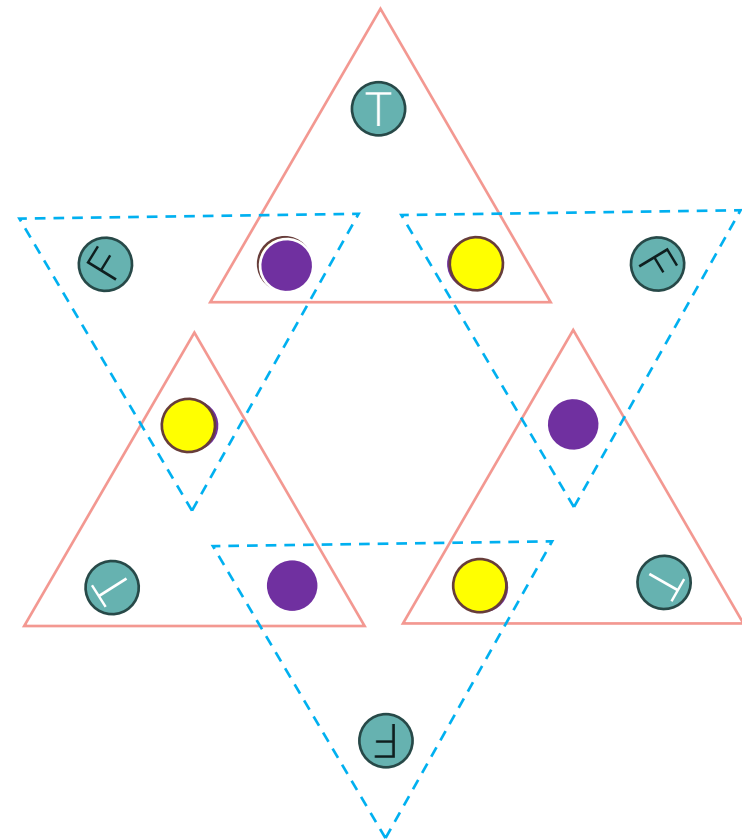
For each variable, we create $4k$ objects.

We designate the tips of the star as T/F, alternating.

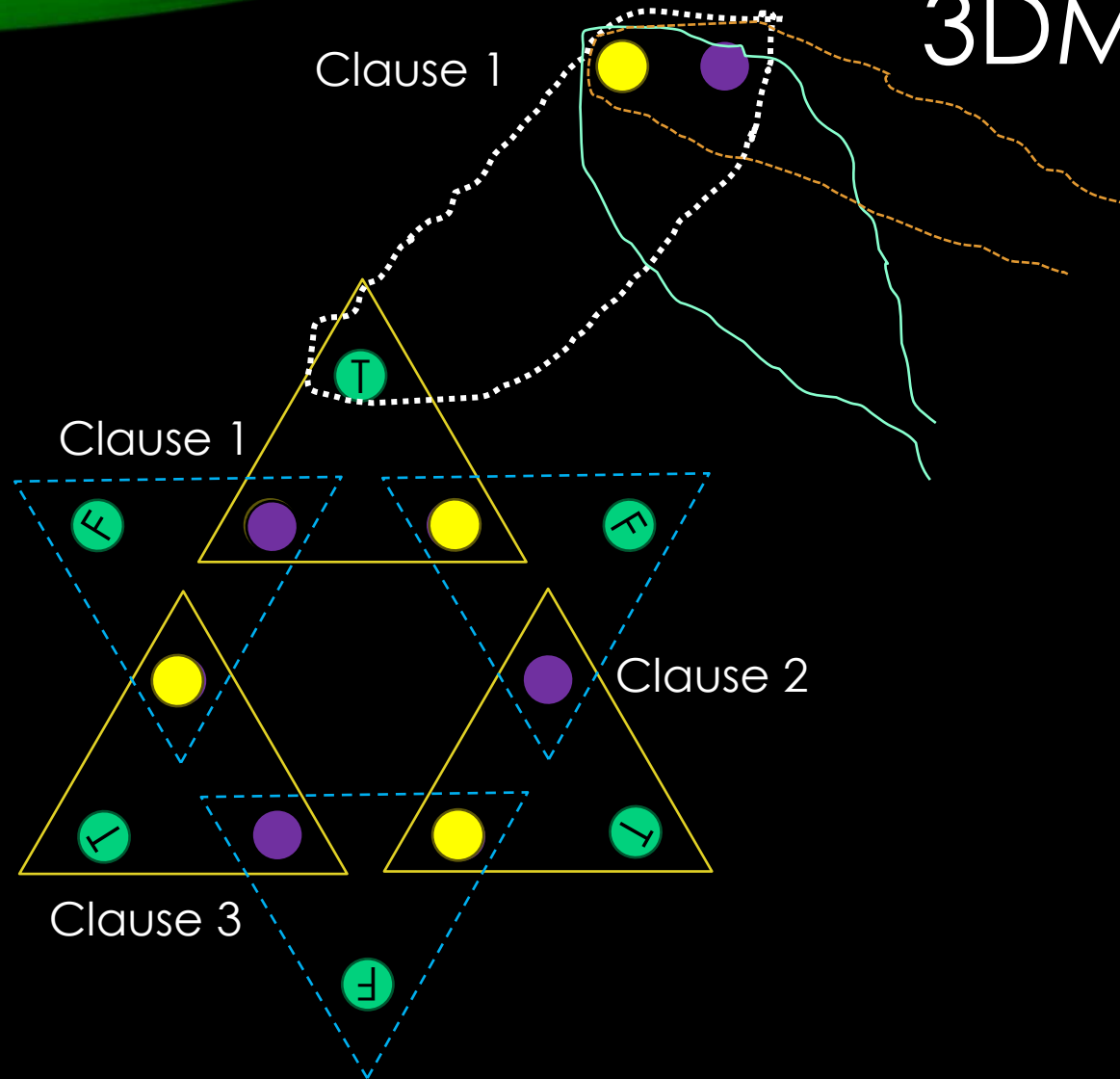
We add triples consisting of each small triangle.

These will be the only triples that include the core elements of the variable gadget.

A perfect matching must take one of the alternating sets. Thus, one of T or F will be left free for each variable.

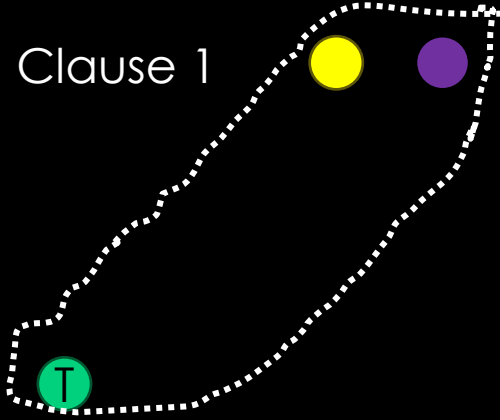


3DM: THE CLAUSE GADGET



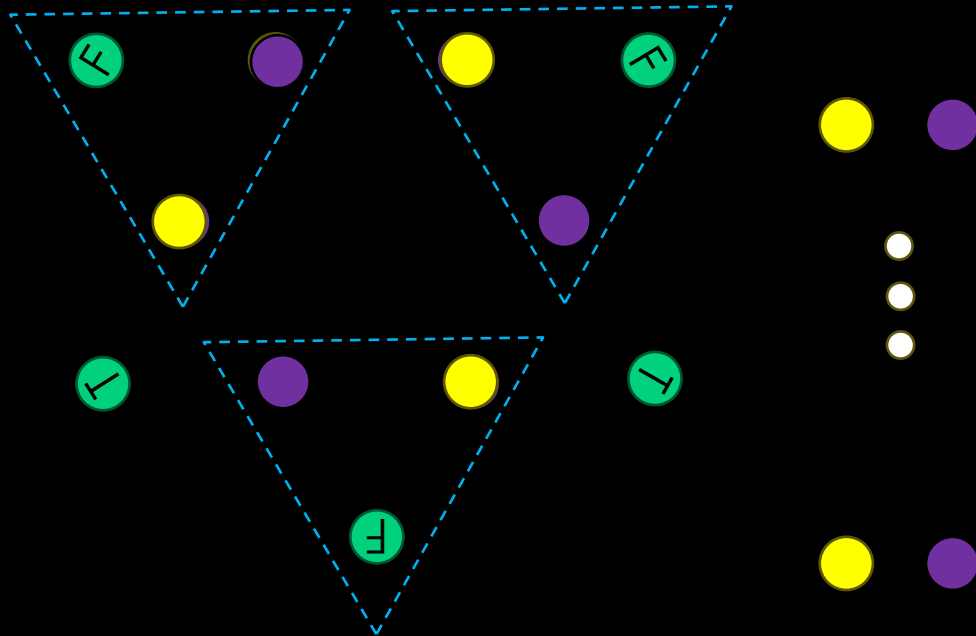
- For each clause, we create just two objects.
 - If a variable appears positively in that clause (i.e. b_i), we create a triple with the associated T element in the variable gadget. If it appears negatively ($\bar{b}_i$), we create a triple with the F element.
 - Do this with all three variables in the clause.
 - Observe that we can only cover the clause objects if one of the three terms is satisfied.

Clause 1

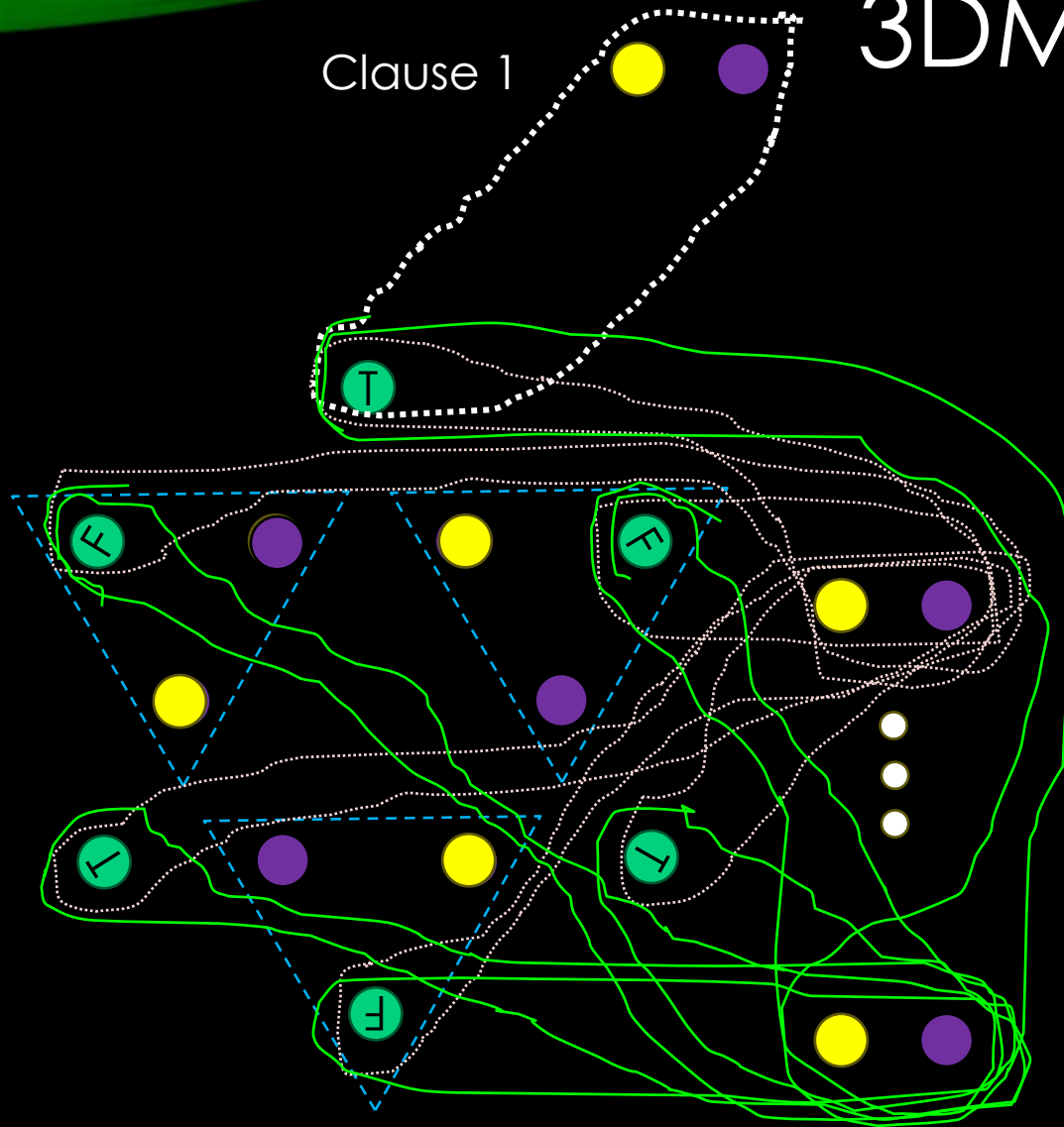


3DM: THE CLEANUP GADGET

- At this point, every satisfying assignment almost produces a perfect matching.
 - But we have some leftover copies of T or F objects that weren't needed by their associated clauses.
 - kn of them are left uncovered by the core triples, and the clauses cover k of them, so there are $k(n - 1)$ left over.
- We create $k(n - 1)$ pairs of “cleanup” objects, and potential triples with each pair and every T/F object.

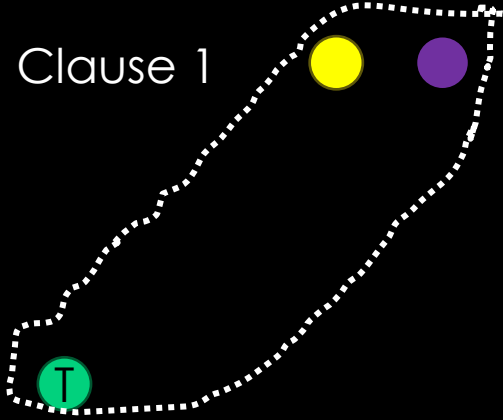


3DM: THE CLEANUP GADGET



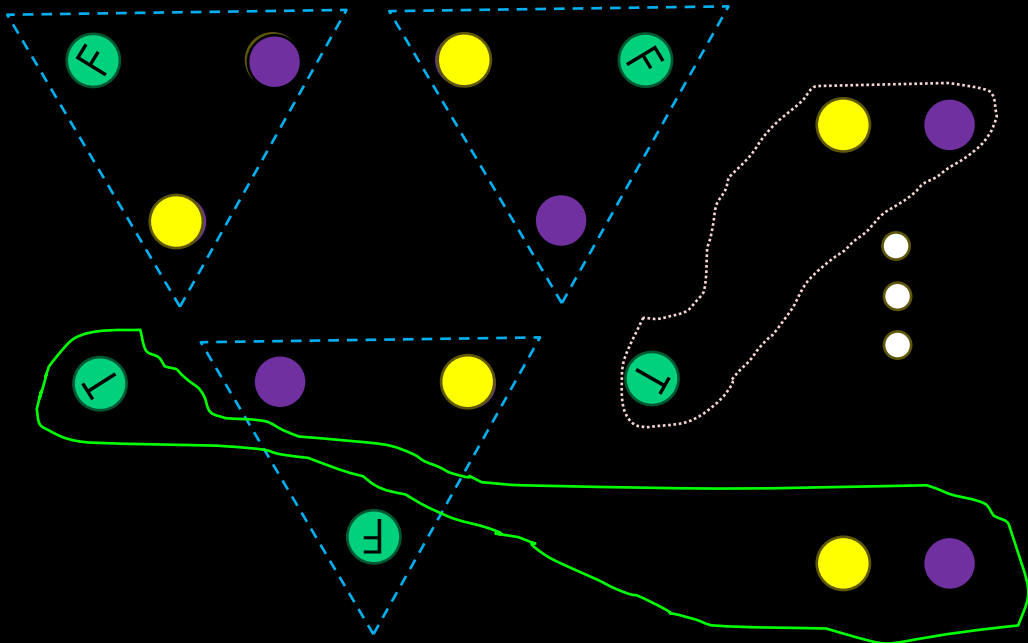
- At this point, every satisfying assignment almost produces a perfect matching.
 - But we have some leftover copies of T or F objects that weren't needed by their associated clauses.
 - kn of them are left uncovered by the core triples, and the clauses cover k of them, so there are $k(n - 1)$ left over.
- We create $k(n - 1)$ pairs of “cleanup” objects, and potential triples with each pair and every T/F object.

Clause 1



3DM: THE CLEANUP GADGET

- The key idea of the cleanup is that we need all three sets (purple, yellow, green) to be the same size.
 - We must connect every cleanup gadget to every tip so that we don't accidentally restrict which variables can be T/F.
- At this point, the 1:1 correspondence between satisfying assignments and 3-D matchings is clear.
- $3\text{-SAT} \leq_p 3\text{DM}$, and 3DM is NP-complete



GRAPH COLORING

The best publicly known instance of this problem is map coloring and the famous Four-Color Theorem.



However, there are lots of more practical uses. Basically any conflict graph leads to a graph coloring problem.

Formally, GC asks: Given a graph $G(V, E)$, can we assign one of k colors to each $v \in V$ such that, for each $(v_i, v_j) \in E$, v_i and v_j are different colors?

2-COLORING VS. 3-COLORING

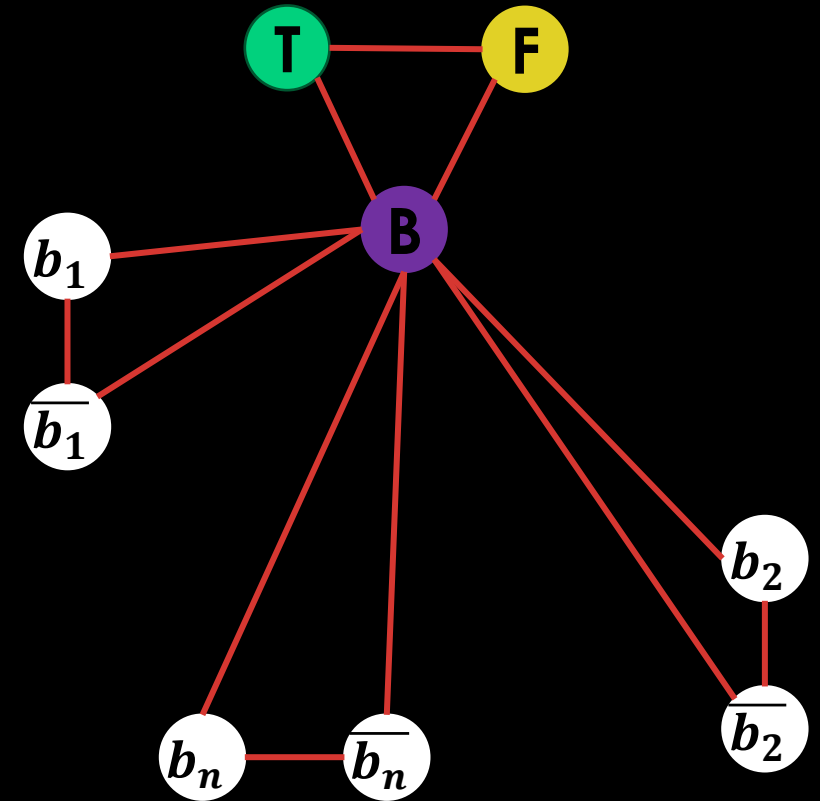
As with matching, $k = 2$ is easy – this just asks whether or not the graph is bipartite, which we can solve using BFS.

... and as with matching, $k = 3$ is hard. We will show that $3\text{-SAT} \leq_p 3\text{-Coloring}$ by building a graph from a 3-SAT instance.

The key advantage here is that Boolean variables cannot be both T and F, and we can model that conflict in a coloring problem.

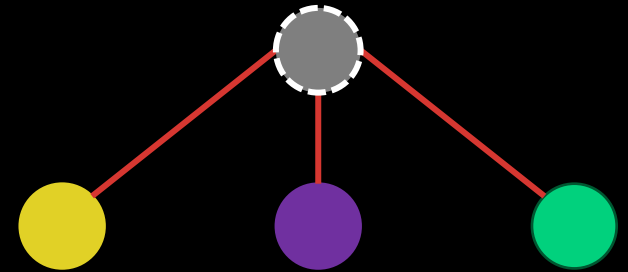
3-COLORING – VARIABLE GADGET

- We'll set up three colors: **True**, **False**, and **Base**.
- Then, for every variable b_i we'll create nodes b_i and $\bar{b}_i$, connecting them to each other and to **Base**.
 - This forces one of each pair to be colored **True** and one to be **False**.
- Observe that every 3-coloring now corresponds to an assignment. We need to use the clause information to force a *satisfying* assignment.



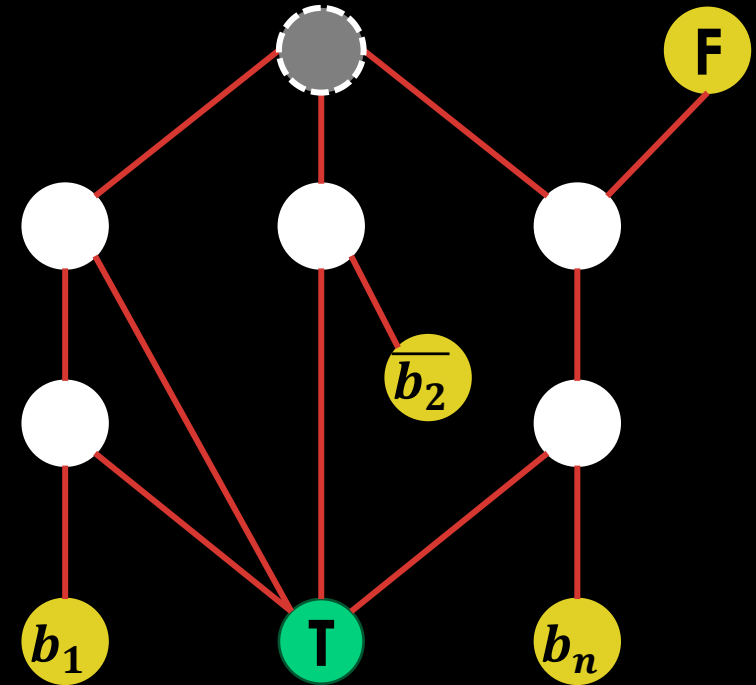
3-COLORING – CLAUSE GADGET

- A clause like $b_1 \vee \overline{b_2} \vee b_n$ translates to “at least one of nodes b_1 , $\overline{b_2}$, or b_n must be assigned the **True** color.”
 - Put another way, we need to make it impossible for a 3-coloring to assign all of them the **False** color.
 - A node is not colorable if it has three different neighbors that are three different colors.
 - Can we force this situation when all three clause nodes are **False**? Yes!



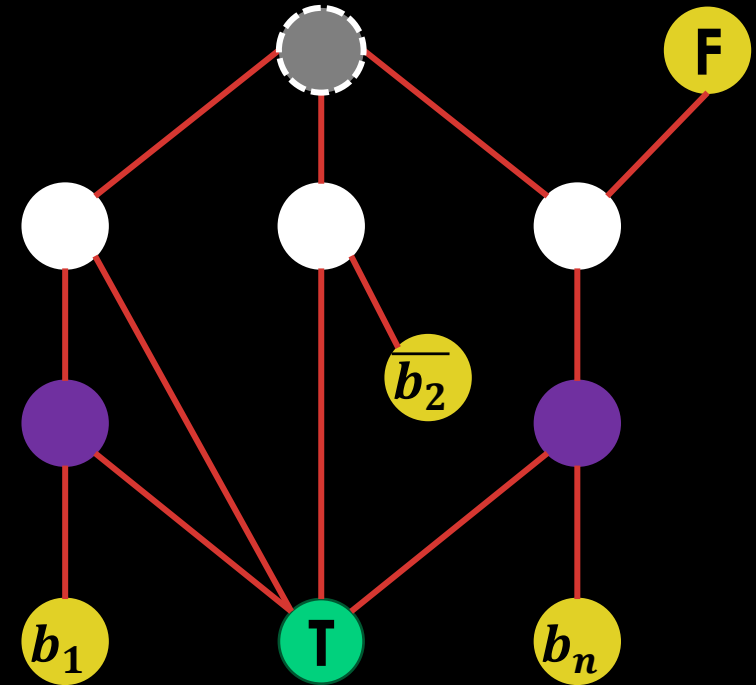
3-COLORING – CLAUSE GADGET

- A clause like $b_1 \vee \overline{b_2} \vee b_n$ translates to “at least one of nodes b_1 , $\overline{b_2}$, or b_n must be assigned the **True** color.”
 - Put another way, we need to make it impossible for a 3-coloring to assign all of them the **False** color.
 - A node is not colorable if it has three different neighbors that are three different colors.
 - Can we force this situation when all three clause nodes are **False**? Yes!



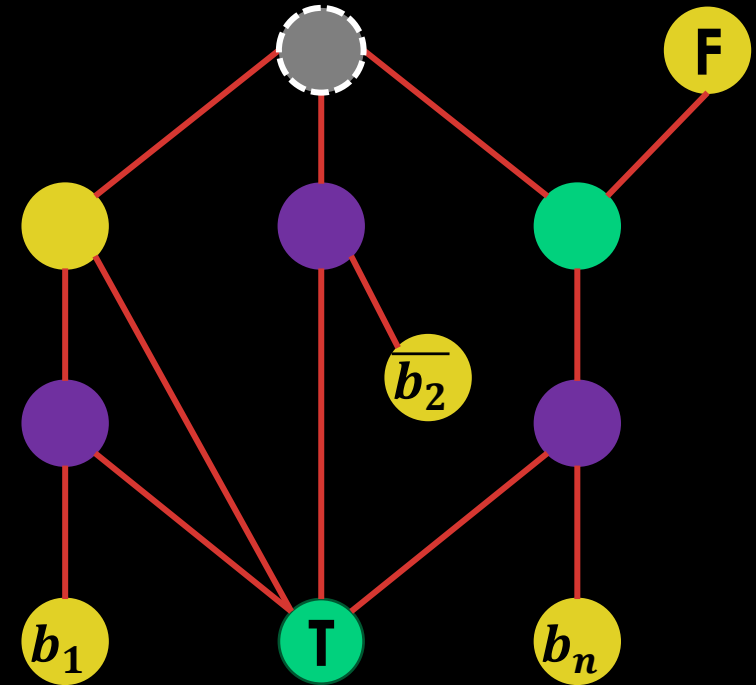
3-COLORING – CLAUSE GADGET

- A clause like $b_1 \vee \overline{b_2} \vee b_n$ translates to “at least one of nodes b_1 , $\overline{b_2}$, or b_n must be assigned the **True** color.”
 - Put another way, we need to make it impossible for a 3-coloring to assign all of them the **False** color.
 - A node is not colorable if it has three different neighbors that are three different colors.
 - Can we force this situation when all three clause nodes are **False**? Yes!



3-COLORING – CLAUSE GADGET

- A clause like $b_1 \vee \overline{b_2} \vee b_n$ translates to “at least one of nodes b_1 , $\overline{b_2}$, or b_n must be assigned the **True** color.”
 - Put another way, we need to make it impossible for a 3-coloring to assign all of them the **False** color.
 - A node is not colorable if it has three different neighbors that are three different colors.
 - Can we force this situation when all three clause nodes are **False**? Yes!



K-COLORING $\leq_p$ 3-COLORING

Any 3-coloring problem can easily be turned into a k -coloring problem for $k > 3$.

Take the graph that you want to 3-color, and add to it a copy of K_{k-3} . (So for 7-coloring, add four nodes all connected to each other.) Now connect every node in that complete subgraph to every node in the original instance.

The new graph is k -colorable if and only if the original graph was 3-colorable.

PONDERABLES...

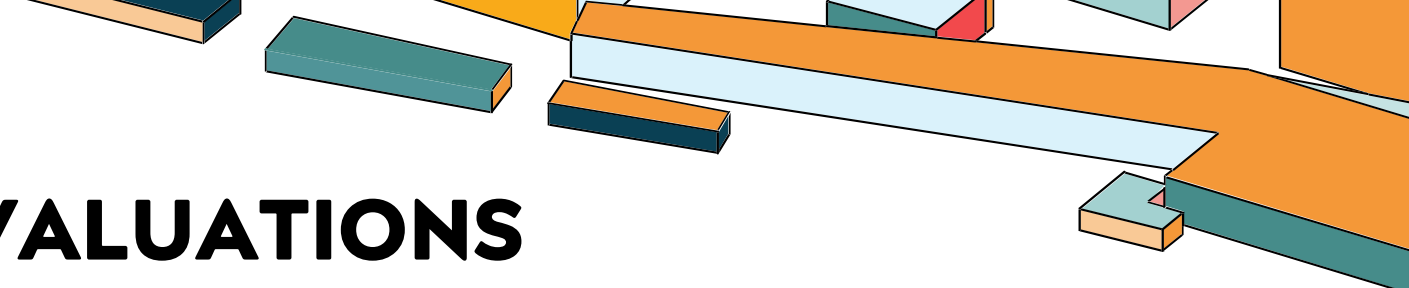
NP -completeness is a very weird thing, and three appears to be a magic number.

- Bipartite matching is in P . 3-D matching is NP -complete.
- 2-coloring is in P . 3-coloring is NP -complete.
- 2-SAT (limiting clauses to two variables) is in P . 3-SAT is NP -complete.
- Eulerian circuit (visit every edge) is in P . Hamiltonian circuit (visit every vertex) is NP -complete.
- Min-cut is in P . Max-cut (or balanced-cut) is NP -complete.
- Linear programming is in P . Integer linear programming is NP -complete.

DEALING WITH NP-COMPLETENESS

You have proven that the problem that you're working on is *NP*-complete. Now what?

- Limit the input size – small cases only!
- Limit the input scope – some problems become tractable when only special cases are allowed.
- Intelligent exponential search (branch-and-bound, backtracking)
- Accept an approximate solution.
- Use a *heuristic* – something that looks good empirically, but has no theoretical guarantees of correctness.
- Spend decades banging your head on the question of $P \stackrel{?}{=} NP$.
- Throw your hands up, tell your boss it can't be done, and launch Steam.



INSTITUTE COURSE EVALUATIONS

- RPI's formal course evaluations ARE NOW OPEN. They will be available through 4/30.
- The department, school, and Institute take these seriously.
- The accuracy of the data depends upon a high participation rate.
- Thus, I will incentivize completion as follows:
 - There are 225 students in the course. 75% is around 170.
 - Each student beyond 170 that completes a course evaluation will result in everyone getting a half-point bonus on their final exam (which will be out of 200 points total).

SUBMITTY POLL

Make sure to submit an answer before leaving.

